DEVELOPER HANDOFF · OPERATIONS · MAINTENANCE

ShopEase PHP Lab

A practical guide for developers who need to diagnose failures, safely fix bugs, test changes, and update the deployed application without prior project knowledge.

Application: plain PHP + PDO, intentionally vulnerable security-training shop
Local database: SQLite
Hosted database: PostgreSQL/Neon when `DATABASE_URL` is configured
Hosting: Render Docker web service
Guide checked against: source at commit `2ddca13`, 22 October 2026

> **Safety boundary.** This application deliberately contains vulnerable behavior. Treat it as a lab, not a production shop. Keep the deployment access gate enabled, use synthetic accounts/data, and only test systems you are authorized to test. Never reset or overwrite Neon data as a troubleshooting shortcut.

Companion source: `docs/DEVELOPER-OPERATIONS-GUIDE.html`
Start with this guide, then check `README.md` for current setup and `VULNERABILITIES.md` for source-reviewed lab behavior.

1 · First response: diagnose before changing

When a page fails, first distinguish an access-gate response from an application or database failure. Do not begin by changing credentials, resetting data, or deploying a speculative fix.

What you see	Likely meaning	Next safe check
ShopEase lab sign-in page	The deployment access gate is active and the browser has not authenticated this session.	Sign in with the authorized lab gate credentials. Never remove the gate to “fix” an access response.
`500`, PHP fatal, SQLSTATE	Application startup, route logic, or database schema/query failed.	Record the exact time, route, deployment commit, and sanitized error. Check Render logs and reproduce locally.
`404`	Wrong route, missing file, or intentionally missing resource.	Verify the URL and file path/case; check whether the record exists in the active database.
Page loads but data is missing	Wrong database/environment, incomplete schema migration, or no matching records.	Verify whether this process is using SQLite or PostgreSQL without printing secrets. Inspect schema and row counts read-only.
Browser says URL required on Delete	Client-side form validation blocked the submit before PHP received it.	Check stored relative image URLs, field types, and the delete button’s `formnovalidate`.

Immediate checks

1. Capture the complete error text and request path. Redact passwords, connection URLs, cookies, tokens, and flag values.
2. Check Render deploy status and identify the exact deployed commit.
3. Check runtime logs around the failure timestamp. Search for PHP fatal errors, `SQLSTATE`, startup messages, and migration/seed failures.
4. Reproduce on a disposable local SQLite database before changing the production database.
5. Make the smallest fix, run targeted tests and PHP lint, then review the complete diff before deployment.
6. Exercise both states of every password visibility control, then verify registration, cart, checkout (including insufficient funds), admin actions, and flag submission.

> **Useful rule:** a successful build or a `401` from the public URL is not an authenticated smoke test. Verify deployment status separately from application behavior.

Do not expose secrets while diagnosing

Never paste `DATABASE_URL`, Render environment dumps, HTTP authorization headers, cookies, superadmin keys, student passwords, or database flag values into issues, logs, screenshots, or chat. Share only the error and sanitized context needed to reproduce it.

PHP warnings and errors are logged by the server and are not displayed in page responses. Diagnose them through authorized, access-controlled server logs; do not turn browser error display on to troubleshoot a deployment.

2 · Architecture and source map

This is a small server-rendered PHP application without a framework. Most requests load `config.php`, which chooses the database, initializes/updates schema, and seeds demo data. In Docker, `auth_gate.php` is configured as PHP's `auto_prepend_file`, so the session-based access check runs before application pages and presents a branded sign-in page when needed.

Path	Responsibility
`config.php`	Vulnerability switches, session setup, database selection/connection, additive schema initialization, demo seed data, challenge-flag catalog, shared helpers.
`auth_gate.php`	Deployment access sign-in using `LAB_AUTH_USER` and `LAB_AUTH_PASS`, protected by a session cookie and CSRF token.
`login.php`, `register.php`	Demo/student authentication and registration. Registration includes the email used for achievement attribution.
`includes/password_toggle.php`	Shared accessible Show/Hide control added to lab access, Superadmin pass-key, learner login, registration, and admin-created-user password fields. Included by the standalone sign-in pages and shared footer.
`products.php`, `api/search.php`, `product.php`	Storefront, search response, product detail and cart entry.
`cart.php`, `checkout.php`, `order_success.php`	Session cart, order confirmation with an order-total summary and delivery form, persisted order display. Insufficient funds are checked when the learner confirms the order, not when adding an item to the cart.
`admin.php`, `pages/`	Admin route and included modules for users, products, orders, and Hall of Heroes.
`superadmin.php`	Separate pass-key-protected control panel and Hall of Heroes summary.
`submit_flag.php`, `generate_cert.php`, `includes/certificate_pdf.php`	Flag submission, earned-challenge certificate authorization, dependency-free PDF generation.
`scripts/reset_demo_data.php`	Explicit, destructive PostgreSQL demo-data reset utility. Not part of normal startup.
`Dockerfile`, `render.yaml`	PHP runtime/extensions, access-gate startup, and Render environment-variable declarations.

Key data relationships

- `users`, `products`, `orders`, and `order_items` contain shop/learner data.
- `flags` holds challenge identifiers, names, and generated flag strings. Do not disclose values.
- `hall_of_fame` holds an award snapshot (user ID, username, email, flag ID, vulnerability ID, timestamp). The submitter self-reports completion; it is not automatic exploit detection.
- `app_metadata` stores one-time initialization markers. Do not remove markers casually; doing so can replay a seed operation.

> Use `VULNERABILITIES.md` as an evidence-based inventory, not as proof that every `VULN_*` toggle has a working challenge. Preserve the lab's explicitly intentional behaviors unless the owner asks for a security-hardening change.

3 · Local development and verification

Requirements

PHP 8.x with `pdo_sqlite` and `pdo_pgsql` for local SQLite and optional PostgreSQL testing. The Render image uses PHP 8.2 CLI and installs both PDO drivers. The local machine's PHP may be newer; test with the deployed runtime when a compatibility issue is suspected.

Start an isolated SQLite copy

```
cd vulnapp-php

APP_DATA_DIR=/tmp/shopease-dev php -S 127.0.0.1:8081
```

Open `http://127.0.0.1:8081`. `APP_DATA_DIR` keeps test data away from the repository's normal local database. Remove only the named disposable test directory when finished. Do not point local experiments at production `DATABASE_URL`.

When `DATABASE_URL` is absent, SQLite is stored at `data/vulnapp.db` (or under `APP_DATA_DIR`). On first initialization, demo users/products and the named challenge catalog are seeded if needed. Demo account details are in the project README; do not reuse them as real credentials.

Minimum verification before a change is accepted

```
# From vulnapp-php/

find . -type f -name '*.php' -print0 | xargs -0 -n1 php -l

git diff --check

git status --short
```

Then exercise the changed flow using a clean temporary SQLite database and synthetic users. For checkout, check both insufficient funds and a successful order. For admin edits, verify the database row changed. For uploads, test valid and rejected files. For certificate changes, check the response with `file certificate.pdf` and verify the award owner check.

Operational behavior to preserve

- Students may add low-balance items to cart; insufficient funds are reported when confirming checkout.
- Cart quantity is at least one; checkout rechecks product and balance.
- Search should be case-insensitive.
- Flag upload accepts plain-text content and does not retain the uploaded file.
- Repeated flag submission must not create duplicate awards for the same learner/flag.
- Admin deletion must work even when an image field contains a relative upload path.

4 · Database, Neon, and flag recovery

Database selection

DATABASE_URL set to a PostgreSQL connection URL selects PostgreSQL/Neon; absent means SQLite. Keep Neon's URL private and configured as a Render secret with SSL. Normal startup runs schema initialization and demo seed checks; it does not intentionally reset shop records.

> **Never run the reset script as a generic repair.** `php scripts/reset_demo_data.php --confirm` permanently deletes shop and learner records in the target PostgreSQL database before reseeding. It is not needed to fix a migration. Verify the target and obtain explicit approval before any destructive database operation.

Challenge flags and the required-ID incident

At startup, `seed_vulnerability_flags()` adds named catalog entries and records a completion marker. Existing deployments may have a legacy `flags.id` column without an automatic ID default; the current code calculates IDs from the existing maximum under a PostgreSQL advisory transaction lock. It should preserve legacy rows and seed only missing challenge names.

If flag seeding fails, the app logs `Vulnerability flag catalog initialization failed: ...` and continues initializing the shop. This is deliberate availability protection: browse/checkout routes can load, but challenge submission may fail until the catalog issue is repaired. A failure before the marker is written may be retried on each request, so inspect logs for repeated errors.

Safe recovery sequence

1. Confirm the route error and corresponding Render log entry. Do not display flags or the Neon URL.
2. Check the existing `flags` and `app_metadata` column definitions and current numeric IDs using a read-only, authorized database session.
3. Back up the database before structural changes. Confirm IDs are numeric and that the unique index on `vulnerability_name` has no conflicting duplicate named rows.
4. Reproduce the legacy schema in disposable SQLite/PostgreSQL test data. Confirm existing flag rows remain, new catalog names get unique IDs, and the second initialization creates no duplicates.
5. Apply the smallest reviewed code/schema fix. Do not truncate tables, rewrite IDs referenced by achievements, drop markers, or reset demo data.
6. After deployment, check logs and an authenticated flag-submission flow. Verify catalog count/award metadata without printing flag values.

Flag submission symptoms

- **Invalid flag:** confirm the same active database contains the flag; local SQLite and Neon are separate stores. Confirm the uploaded file is plain text and within the size limit.
- **PDF not authorized/not found:** the certificate route requires a signed-in learner who owns the award. Check `hall_of_fame.user_id` and `flag_id`.
- **Missing Hall of Heroes email:** legacy rows may predate the email snapshot; new registration/submission stores email going forward. Do not invent email values for old records.

5 · Render operation and safe deployment

Deployment boundary

The Render service is named `vulnapp-php` and builds the PHP Docker image. The Docker command starts PHP's built-in web server and prepends `auth_gate.php`. Keep these Render variables configured as secrets: `LAB_AUTH_USER`, `LAB_AUTH_PASS`, `SUPERADMIN_KEY`, and `DATABASE_URL`. The access gate now shows a ShopEase sign-in page and keeps a session after successful authentication; do not remove the gate. Never place secret values in `render.yaml`, commits, or this guide.

The locally configured Git remote is the source connected to the deployment. Check `git remote -v` and Render's service settings before publishing; workspace display names are not necessarily GitHub repository names.

Release checklist

1. Inspect `git status`. Stage only the files for this task; this workspace can contain unrelated changes.
2. Run PHP lint across the app and `git diff --check`. Complete local, isolated smoke tests.
3. Review the final diff, especially schema initialization, database writes, upload handlers, and admin routes.
4. Commit with a clear message and push to the verified deployment branch.
5. Trigger a deploy for that exact commit (or confirm auto-deploy), then wait for Render status `live`.
6. Check startup/runtime logs. Confirm the public URL responds. A 401 only confirms the access gate is answering; use authorized credentials to exercise the protected app.
7. Confirm a small end-to-end flow and monitor logs after the release. Report the commit and any access limitation accurately.

Render CLI examples

```
# Inspect latest deployments

render deploys list SERVICE_ID --output json



# Inspect recent logs (avoid sharing secrets in output)

render logs --resources SERVICE_ID --limit 100 --output json



# Deploy a reviewed, pushed commit and wait for completion

render deploys create SERVICE_ID --commit COMMIT_SHA --wait --confirm --output json
```

Use the actual service ID from the Render dashboard/CLI; do not assume it from an old session. Restarting is not a substitute for fixing a failing migration. Keep Neon records intact during deploys.

Rollback

If the new release causes an outage, identify the last known-good deployment in Render. Roll back to that deployment or redeploy the previous known-good commit. A code rollback does not undo database changes; keep migrations additive and backward-compatible, and handle database rollback separately with a backup and review.

> A rollback, redeploy, or restart should not print or alter database credentials, reset Neon, or expose the intentionally vulnerable service by removing the access gate.

6 · Common problems and developer handoff

Symptom	Checks and fix direction
Fatal mentions SQLSTATE `25P02`	PostgreSQL is reporting a command after an earlier query already aborted the transaction. Find the first database error in logs; later errors may be secondary. Roll back the transaction on failure and avoid continuing queries inside an aborted transaction.
Null value violates `flags.id`	Legacy table may require explicit IDs. Use the additive catalog seeder's max-ID logic under a transaction lock; test numeric legacy IDs and uniqueness. Do not drop the primary key/default or clear rows as a shortcut.
Shop loads, flags do not	Flag initialization is best-effort to preserve shop availability. Look for the catalog initialization error in Render logs; repair catalog/schema before claiming flag submissions work.
Product image does not survive deploy	Container-local `uploads/` is not durable on Render. Use configured persistent file storage; the URL/local upload form does not itself provide persistence.
PDO driver missing locally	Install the matching PHP SQLite/PostgreSQL PDO extension in the developer environment. The Docker image installs `pdo_sqlite` and `pdo_pgsql`.
Admin add/edit triggers header warning	Admin modules are included after the page header is emitted. Do not call `header()` from the included module at that stage; render a success state or move redirect handling before output.
Delete asks for an image URL	Check HTML form association and browser validation. Keep the delete button's `formnovalidate` and accept relative stored upload paths in the image field.

Before handing the change to another developer

- State the user-visible symptom, exact failing route, and the minimal steps to reproduce.
- Provide the commit SHA, deployment status, tests run, and whether the verification used SQLite or Neon.
- Summarize any schema change, whether it is additive, and how existing rows are preserved.
- Include relevant sanitized logs; remove secrets, cookies, personal data, and challenge flag strings.
- List unresolved checks plainly. Do not say "fixed" if only a build or unauthenticated 401 check passed.

> **Intentional lab vulnerabilities.** Do not "clean up" vulnerable handlers or disable challenge switches as part of an unrelated bug fix. Conversely, do not expose this lab publicly without the access gate. Security hardening should be a separately scoped, owner-approved change.

This document describes the PHP application at the source revision shown on the cover. Recheck the README, source, Render settings, and current deployment after future changes; settings and services can change independently of this guide.